FINTECH PAYMENT SYSTEM

System Architecture Document

Mobile Financial Service — bKash-like Platform

Academic Sessional — System Design at Scale

1. System Overview

This document describes the system architecture for a mobile financial service (MFS) platform implementing OTP-based login, wallet balance check, and peer-to-peer money transfer — equivalent to a subset of bKash. The architecture is designed for real-world scale principles while avoiding unnecessary complexity for the current feature set.

Core features implemented: OTP login, check balance, send money
Stack: Node.js + Express (backend), React.js (frontend), PostgreSQL, Redis, RabbitMQ
Target: Production-grade architecture taught at beginner-to-intermediate level
Services: Account Service, Wallet & Transaction Service, OTP Service, API Gateway

2. Service Architecture

The system is decomposed into three backend services plus an API gateway. Each service boundary is drawn along lines of independent deployability, independent failure domains, and distinct data ownership. The separation is not arbitrary — each split is justified below.

2.1 Account Service (with Auth)

Owns all user identity, authentication, profile management, and JWT issuance. Auth lives here — not in a separate auth service — because auth is fundamentally about user identity and there is only one consumer application.

Responsibilities:

- User registration and profile management
- JWT access token issuance and refresh token management
- Favourite numbers (saved contacts) management

- Account status management (active, suspended)
- Calls OTP Service during login — synchronous HTTP

Why separated from Wallet & Transaction Service:

- Different change rate: user identity data changes rarely; transaction data changes on every send
- Different scale requirement: auth/profile reads are low volume; transaction writes are high volume and need independent scaling
- Independent failure domain: if Wallet Service is degraded, users can still log in and view profile
- Different compliance boundary: PII (name, phone) is regulated differently from financial transaction records
- Security isolation: JWT signing secret and user credentials live only here, not exposed to financial service

Why auth is NOT a separate service:

- Only one application authenticates — no cross-app SSO requirement
- Auth logic is 3 functions: issue JWT, verify OTP, refresh token — not enough to justify a separate deployable
- Separate auth service adds a network hop on every login with zero benefit at this scale

2.2 Wallet & Transaction Service

Owns all financial operations. Balance, send money, transaction history, and external disbursements. Wallet and transaction are deliberately kept in the same service because they share an atomic database transaction on every money movement — splitting them would require a distributed transaction (saga or 2PC), which is one of the hardest problems in distributed systems.

Responsibilities:

- Wallet creation, balance management
- Internal transfers: send money, bill pay, merchant payment
- External disbursements: mobile recharge, send to bank
- Transaction history and statement generation
- Receiver wallet lookup and validation
- Publishes domain events on every transaction outcome

Why Wallet and Transaction are NOT split:

- A send-money operation reads wallet balance and writes a transaction record in one atomic BEGIN…COMMIT block — splitting them makes this a distributed transaction
- Distributed transactions require saga orchestration or 2-phase commit — serious overkill and a known source of production bugs
- They scale identically: every transaction touches both a wallet row and a transaction row
- They fail identically: if the DB is down, neither can function

- The coupling is intentional and correct — these two concerns are genuinely inseparable

Why separated from Account Service:

- Financial data has stricter audit and compliance requirements than profile data
- Transaction volume is orders of magnitude higher than profile reads — needs independent horizontal scaling
- Wallet service can be deployed with read replicas for balance checks without affecting Account Service
- Data isolation: wallet schema is accessible only by Wallet Service DB credentials — Account Service cannot read financial data

2.3 Notification Service

The only service justified as truly standalone. Handles transactions SMS delivery, OTP generation, storage, delivery via SMS gateway, and verification. Uses RabbitMQ for async SMS delivery.

Responsibilities:

- Generate cryptographically random 6-digit OTP
- Store OTP in Redis with 5-minute TTL
- Enqueue SMS delivery job to RabbitMQ
- Worker process consumes queue and calls SMS gateway
- Transaction confirmation SMS delivery
- Verify OTP: direct Redis lookup, synchronous, under 5ms
- Rate limiting: max 3 OTP requests per phone per 10 minutes
- Attempt limiting: invalidate OTP after 3 wrong attempts

Why separated from Account Service:

- SMS provider is a swappable external dependency — today Twilio/SSL Wireless, tomorrow a cheaper local BD provider — this swap requires zero changes to Account Service
- OTP delivery has its own failure mode: SMS gateway outages should not crash Account Service
- The queue (RabbitMQ) absorbs SMS delivery spikes independently — 10,000 simultaneous login requests enqueue jobs without blocking the account service
- OTP service has no knowledge of users — it only knows phone numbers and codes — this tight scope makes it independently testable and replaceable
- Retry logic for failed SMS delivery belongs here, not scattered in Account Service

2.4 API Gateway (Nginx)

Single entry point for all client traffic. Routes requests to appropriate services, enforces rate limiting, handles SSL termination, and provides a unified interface to the Next.js frontend.

Responsibilities:

- SSL/TLS termination — HTTPS enforced at the edge, internal traffic can be HTTP
- Request routing to services by path prefix
- Global rate limiting — 100 requests per IP per minute across all endpoints
- Request logging at the network level
- Load balancing across multiple instances of each service

3. Database Architecture

3.1 PostgreSQL — Schemas and Ownership

One PostgreSQL instance with separate schemas per service. Each service connects with its own DB credentials that have access only to its own schema. This enforces service boundary at the database level — not just by code discipline.

Schema	Owned By	Tables	DB User
account_db	Account Service	users	account_svc_user
wallet_db	Wallet & Txn Service	wallets, transactions, audit_logs	wallet_svc_user
operational_db	Shared (write-async)	operational_logs	ops_log_user

3.2 Schema Definitions

account_db.users

```
id            UUID PRIMARY KEY DEFAULT gen_random_uuid()
phone         VARCHAR(20) UNIQUE NOT NULL
name          VARCHAR(100) NOT NULL
pin_hash      VARCHAR(255)  -- bcrypt hashed PIN
status        VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active','suspended','closed'))
created_at    TIMESTAMPTZ DEFAULT now()
updated_at    TIMESTAMPTZ DEFAULT now()
```

wallet_db.wallets

```
id            UUID PRIMARY KEY DEFAULT gen_random_uuid()
user_id       UUID NOT NULL        -- logical ref to account_db.users.id, NO FK CONSTRAINT
phone         VARCHAR(20) NOT NULL  -- denormalized copy from Account Service at creation
display_name  VARCHAR(100)         -- denormalized copy, updated via events
balance       NUMERIC(15,2) DEFAULT 0 CHECK (balance >= 0)
```

```
wallet_type     VARCHAR(20) DEFAULT 'personal' CHECK (wallet_type IN
('personal','merchant','system'))
status          VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active','suspended','closed'))
currency        VARCHAR(3) DEFAULT 'BDT'
created_at      TIMESTAMPTZ DEFAULT now()
```

wallet_db.transactions

```
id                  UUID PRIMARY KEY DEFAULT gen_random_uuid()
sender_wallet_id    UUID REFERENCES wallet_db.wallets(id)
receiver_wallet_id  UUID REFERENCES wallet_db.wallets(id)
amount              NUMERIC(15,2) NOT NULL CHECK (amount > 0)
txn_type            VARCHAR(20) DEFAULT 'internal' CHECK (txn_type IN ('internal','external'))
status              VARCHAR(20) DEFAULT 'completed' CHECK (status IN
('pending','completed','failed','refunded'))
idempotency_key     VARCHAR(100) UNIQUE
provider            VARCHAR(50)    -- 'robi', 'beftn', null for internal
provider_ref        VARCHAR(100)   -- provider's transaction ID for reconciliation
metadata            JSONB
created_at          TIMESTAMPTZ DEFAULT now()

INDEX ON transactions (sender_wallet_id, created_at DESC)
INDEX ON transactions (receiver_wallet_id, created_at DESC)
```

wallet_db.audit_logs

```
id          UUID PRIMARY KEY DEFAULT gen_random_uuid()
event_type  VARCHAR(50) NOT NULL  -- 'TRANSFER_COMPLETED', 'OTP_REQUESTED', etc.
actor_id    UUID                  -- user who performed the action
target_id   UUID                  -- affected entity
metadata    JSONB                 -- amount, wallet IDs, provider ref, etc.
created_at  TIMESTAMPTZ DEFAULT now()

-- Never deleted. Written inside same DB transaction as the financial event.
-- Consistent by DB guarantee: if transfer rolls back, audit log rolls back too.
```

3.3 Cross-Schema Relationship Rules

The relationship between account_db and wallet_db is logical — enforced by the application, not by the database engine. Three hard rules apply:

- No cross-schema FOREIGN KEY constraints — ever. wallets.user_id is a UUID value, not a DB-level foreign key reference to users.id

- No service reads another service's schema — enforced by separate DB credentials at the Postgres level
- Data needed across schemas is either denormalized at write time (phone, display_name on wallets) or requested via API at runtime

Why no cross-schema FK?
A FK constraint couples the schemas at the DB level. If Account Service needs to rename a column, drop a table, or migrate the users table — the FK in wallet_db breaks. Two teams, one hidden coupling, production incident. The logical reference is maintained by the application and events.

4. Redis — Caching and Ephemeral State

Redis serves four distinct purposes in this system. Each purpose uses a different key namespace and TTL strategy.

Key Pattern	Value	TTL	Purpose	Owner
otp:{phone}	{ code, attempts }	300s	OTP code storage	OTP Service
otp_requests:{phone}	integer counter	600s	OTP rate limiting	OTP Service
refresh:{userId}:{tokenId}	{ token, ip, ua }	7 days	Refresh token store	Account Service
rate:{ip}	integer counter	60s	Global rate limiting	API Gateway
session_blacklist:{jti}	1	15 min	Revoked token tracking	Account Service

4.1 OTP Storage

OTP codes are stored in Redis with a 5-minute TTL. The value includes the code and attempt count. On correct entry the key is deleted immediately — one-time use enforced at the data layer, not just application logic. On wrong entry the attempt count is incremented with KEEPTTL to preserve the original expiry.

4.2 Refresh Token Management

Refresh tokens are opaque random strings stored in Redis. Each user can have multiple refresh tokens — one per device. Logout deletes the specific token. Logout-all deletes all keys matching refresh:{userId}:*. Token theft detection: if the same refresh token is presented twice, all sessions for that user are immediately revoked.

4.3 RabbitMQ — OTP and SMS Queue

RabbitMQ is used specifically for SMS delivery in the Notification Service. The Account Service publishes a send-otp job to the queue and returns 200 to the client immediately. A separate worker process consumes the queue, generates the OTP, stores it in Redis, and calls the SMS gateway. RabbitMQ provides retry logic on SMS gateway failures with exponential backoff — up to 3 retries before moving to a dead-letter queue.

```
Queue: otp.send
Dead-letter queue: otp.send.dlq  (failed after 3 retries)
Retry: exponential backoff — 5s, 15s, 45s
Worker: separate Node.js process, horizontally scalable
Why RabbitMQ not Redis pub/sub: RabbitMQ persists jobs to disk.
If the worker crashes mid-delivery, the job is not lost. Redis pub/sub is fire-and-forget.
```

5. Security Architecture

Security is layered — each layer defends against a different attack surface. No single layer is sufficient alone.

5.1 Transport Security

- HTTPS enforced at API Gateway (Nginx) — all client traffic encrypted with TLS 1.2+
- HTTP Strict Transport Security (HSTS) header set — browsers refuse non-HTTPS connections
- Internal service-to-service communication over private network — HTTP acceptable internally
- SSL certificates: Let's Encrypt with auto-renewal via Certbot

5.2 Authentication & Token Security

- JWT access tokens: signed with HS256, 15-minute expiry — short enough that theft is time-limited
- Refresh tokens: opaque random UUID, stored in Redis, 7-day TTL, revocable instantly
- JWT payload: only userId, phone, iat, exp, jti — no balance, no wallet ID, no sensitive data
- Token rotation: every refresh issues a new refresh token and revokes the old one
- Concurrent session detection: presenting a used refresh token triggers full session revocation
- Logout: deletes refresh token from Redis — JWT becomes orphaned and unusable for refresh

5.3 OTP Security

- Rate limiting: max 3 OTP requests per phone per 10 minutes — stored in Redis with TTL
- Attempt limiting: 3 wrong attempts invalidates the OTP — must request a new one
- OTP never logged: audit log records that OTP was sent, never the code itself
- One-time use: Redis key deleted immediately on successful verification
- 5-minute TTL: expired codes rejected at Redis layer before application code runs

5.4 API Security (Applied at API Gateway + Each Service)

- Helmet.js on every Express service: sets Content-Security-Policy, X-Frame-Options, X-Content-Type-Options headers
- express-rate-limit: per-endpoint limits in addition to global gateway rate limiting
- express-validator: input validation and sanitization on every route that accepts body data
- CORS: configured to accept only the known frontend origin — no wildcard
- SQL injection: parameterized queries only — no string concatenation in SQL ever

5.5 Financial Transaction Security

- SELECT FOR UPDATE on sender wallet row — prevents race condition where two concurrent requests both pass the balance check
- Idempotency key: unique constraint in DB — client can safely retry failed requests without double-charging
- Amount validation: server-side only — client-provided amounts are never trusted
- Balance floor: CHECK (balance >= 0) at DB level — cannot go negative even if application logic fails
- Atomic transactions: BEGIN…COMMIT wraps debit, credit, and audit log — no partial state possible

5.6 Security Layer Map

Layer	What It Protects	Applied Where
TLS/HTTPS	Data in transit	Nginx API Gateway
JWT + Refresh	Session security and revocation	Account Service
OTP rate limit	Brute force on login	OTP Service + Redis
Helmet.js	HTTP header attacks	All Express services
Rate limiting	DDoS and abuse	Nginx + express-rate-limit
Input validation	Injection, malformed data	All routes via express-validator
SELECT FOR UPDATE	Race conditions on balance	Wallet & Txn Service
Idempotency key	Duplicate transactions	Wallet & Txn Service DB

Layer	What It Protects	Applied Where
DB CHECK constraints	Data integrity floor	PostgreSQL schema

6. Logging Strategy

Logging is split into two categories with fundamentally different requirements: audit logs (permanent, consistent, financial compliance) and operational logs (ephemeral, best-effort, developer observability).

6.1 Audit Logs — What to Log and Where

Written inside the same database transaction as the financial event. If the transaction rolls back, the audit log rolls back too. Never deleted. Stored in wallet_db.audit_logs.

Event	Produced By	Key Metadata
USER_REGISTERED	Account Service	user_id, phone
OTP_REQUESTED	OTP Service	phone, ip_address
OTP_VERIFIED	OTP Service	phone, user_id
OTP_FAILED	OTP Service	phone, attempt_count
LOGIN_SUCCESS	Account Service	user_id, ip, user_agent
TRANSFER_INITIATED	Wallet & Txn Service	sender_id, receiver_id, amount, idempotency_key
TRANSFER_COMPLETED	Wallet & Txn Service	txn_id, amount, new_sender_balance
TRANSFER_FAILED	Wallet & Txn Service	reason, was_debited
EXTERNAL_PAYMENT_INITIATED	Wallet & Txn Service	provider, amount, reference
EXTERNAL_PAYMENT_COMPLETED	Wallet & Txn Service	provider_ref, status
TRANSFER_REFUNDED	Wallet & Txn Service	original_txn_id, reason
ACCOUNT_SUSPENDED	Account Service	user_id, reason, suspended_by

6.2 Operational Logs — What to Log and Where

Written asynchronously via an in-memory buffer that flushes to DB in batches every 5 seconds. Never blocks a request. Purged after 30 days. Used for debugging and performance monitoring.

- HTTP request/response: method, path, status code, latency ms, user_id — every request, via middleware
- DB slow queries: any query over 200ms — inside repository layer
- External API calls: SMS gateway, BEFTN — provider, latency, status code
- External API failures: full error message (never response body — may contain PII)
- Queue events: job enqueued, job consumed, job failed, retry count
- Service startup/shutdown: for deployment tracking
- Unhandled exceptions: message + stack trace

6.3 What NOT to Log

- OTP code values — log that OTP was sent, never the code itself
- JWT token content — log that login succeeded, never the token
- Passwords or PINs — never, anywhere
- Full wallet balances in operational logs — balance belongs only in audit logs with stricter access
- Full external API response bodies — may contain sensitive provider data

7. Deployment Architecture

Docker Compose for local development and single-server staging. Docker + manual orchestration (or lightweight Kubernetes) for production. For an academic project, Docker Compose on a single VPS is sufficient to demonstrate the architecture.

7.1 Docker Setup

Each service has its own Dockerfile. Docker Compose orchestrates all services locally. The compose file defines the service graph, shared network, and volume mounts.

```
Services in docker-compose.yml:
  nginx           — API Gateway, ports 80/443
  account-service — Node.js, internal port 3001
  wallet-service  — Node.js, internal port 3002
  otp-service     — Node.js, internal port 3003
  otp-worker      — Node.js worker (RabbitMQ consumer, no HTTP port)
  postgres        — PostgreSQL 15, port 5432 (internal only)
  redis           — Redis 7, port 6379 (internal only)
  rabbitmq        — RabbitMQ 3 with management UI, port 15672
  frontend        — Next.js, port 3000
```

7.2 Production Deployment (Single VPS — Entry Level)

- Single VPS: 4 vCPU, 8GB RAM (DigitalOcean, Hetzner, or AWS EC2 t3.medium)
- Docker Compose in production — sufficient for academic demonstration and low-traffic launch
- Nginx on host machine handles SSL termination and proxies to Docker containers
- PostgreSQL and Redis run in containers with named volumes for data persistence
- Each service container limited to 512MB RAM to prevent one service starving others

7.3 Scaling to Multi-Server (Growth Path)

When the single VPS is no longer sufficient, the natural scaling path is:

- Horizontal scaling: run multiple instances of Account Service and Wallet Service behind Nginx upstream load balancing — sessions are in Redis so any instance can serve any request
- DB read replica: add a PostgreSQL read replica for balance checks and transaction history — writes go to primary, reads go to replica
- Managed Redis: move to Redis Cluster or ElastiCache for HA Redis
- Kubernetes: if team size grows and independent deployment frequency increases, migrate to K8s — but only then

On Kubernetes vs Docker Compose:
Kubernetes is justified when: multiple teams deploy independently, you need automated rollbacks, auto-scaling on CPU metrics, or complex canary deployments.
For a 3-service app with one team: Docker Compose is not a compromise — it is the correct tool. Running K8s for 3 services adds 40+ hours of ops overhead for near-zero benefit.

8. Network Components

Component	Tool	Count	Purpose
API Gateway / Reverse Proxy	Nginx	1	SSL termination, routing, rate limiting
Load Balancer	Nginx upstream	1 (built into gateway)	Round-robin across service instances
CDN	Cloudflare (free tier)	1	Static asset caching, DDoS protection, global DNS
Internal Network	Docker bridge	1	Service-to-service communication, not exposed
Message Broker	RabbitMQ	1	OTP delivery queue with persistence

Component	Tool	Count	Purpose
			and retry

8.1 CDN — Cloudflare

- Sits in front of Nginx — all traffic hits Cloudflare first
- Caches Next.js static assets (JS bundles, images) at edge — reduces origin server load
- Free DDoS protection layer — absorbs volumetric attacks before they hit your VPS
- Manages DNS — easy to point traffic to a new server IP without client-visible downtime
- Free SSL certificate as backup to Let's Encrypt

8.2 What You Do NOT Need

- Separate hardware load balancer — Nginx handles this in software
- Multiple CDN providers — Cloudflare free tier is sufficient
- Service mesh (Istio, Linkerd) — adds massive operational complexity for 3 services
- Multiple RabbitMQ nodes / clustering — single node with durable queues is sufficient

9. Post-Live Observability

Four tools cover the full observability stack: uptime monitoring, error tracking, infrastructure metrics, and application performance. Each has a distinct role — they are not redundant.

9.1 UptimeRobot — Availability Monitoring

Pings your service endpoints every 5 minutes from external servers. Sends alerts via SMS/email if a service is unreachable. Free tier covers 50 monitors.

- Monitor: API Gateway health endpoint — GET /health → 200
- Monitor: Account Service — GET /account/health
- Monitor: Wallet Service — GET /wallet/health
- Monitor: Frontend — GET / returns 200
- Alert: SMS + email to on-call developer within 5 minutes of downtime

Why: UptimeRobot tells you your service is down before your users do. It is the first line of alert.

9.2 Sentry — Error Tracking

Captures unhandled exceptions and promise rejections across all Node.js services in real time. Groups errors by type, shows stack traces, tracks error frequency, and links errors to specific releases.

- Installed in every Express service: one line — Sentry.init({ dsn: process.env.SENTRY_DSN })

- Captures: unhandled exceptions, failed DB queries, failed external API calls
- Does NOT capture: expected business errors (insufficient balance, wrong OTP) — those are handled flows
- Release tracking: tag each deployment with git commit SHA — Sentry shows which release introduced a bug
- Alerts: email when a new error type appears or error rate spikes above threshold

Why: Sentry catches errors that don't cause downtime but do cause bad user experience — a specific endpoint 500ing for a subset of users that UptimeRobot would never detect.

9.3 Grafana + Prometheus — Infrastructure and Application Metrics

Prometheus scrapes metrics from all services every 15 seconds. Grafana visualizes them as dashboards. Together they answer 'how is the system behaving right now and over time.'

- Infrastructure metrics (via Node Exporter): CPU usage per server, memory usage, disk I/O, network throughput
- Application metrics (via prom-client in each service): request latency per endpoint, requests per second, error rate, DB connection pool usage
- Business metrics: transactions per minute, OTP send rate, failed transfer rate
- Alerts in Grafana: alert when p99 latency on /transaction/send exceeds 2 seconds, or error rate exceeds 1%

Why: UptimeRobot tells you if the service is up or down. Grafana tells you if it is slow, degraded, or approaching a resource limit before it goes down.

9.4 Netdata — Real-Time System Monitoring

Installed directly on the VPS host. Provides real-time (per-second) visibility into CPU, RAM, disk, and network at the OS level — finer granularity than Prometheus for immediate incident diagnosis.

- Install: one command — curl https://my-netdata.io/kickstart.sh | bash
- Useful for: identifying which Docker container is consuming runaway CPU during a spike
- Complements Grafana: Grafana shows trends over time, Netdata shows what is happening right now

Why: During a live incident, Netdata tells you in 5 seconds whether the problem is the DB, the OTP worker, or the network interface.

9.5 Observability Tool Summary

Tool	What It Tells You	Granularity	Alert Capable
UptimeRobot	Is the service reachable at all?	5 min	Yes — SMS/email
Sentry	What errors are users hitting?	Real-time	Yes — on new/spike
Grafana + Prometheus	How is performance trending?	15 sec	Yes — threshold

Tool	What It Tells You	Granularity	Alert Capable
Netdata	What is consuming resources right now?	1 sec	Yes — basic

10. Capacity and Load Estimation

This section estimates how much traffic the described architecture can handle at each deployment tier. Estimates are based on typical Node.js throughput on standard hardware with PostgreSQL and Redis.

10.1 Single VPS — 4 vCPU, 8GB RAM

Operation	Estimated Throughput	Bottleneck
OTP requests	~200 req/sec	SMS gateway rate limit, not your server
Login (verify OTP)	~500 req/sec	Redis lookup — very fast
Check balance	~800 req/sec	DB read with index — fast
Send money	~150 req/sec	Postgres SELECT FOR UPDATE + COMMIT — serialized per wallet
Transaction history	~600 req/sec	DB read with index

In practice: a single VPS handles approximately 50,000–100,000 daily active users comfortably at typical usage patterns (users do not all transact simultaneously). Peak load during Eid or payday could 5–10x normal traffic — this is where horizontal scaling of the Wallet Service and a DB read replica become necessary.

10.2 Horizontal Scale — 3 Service Instances + Read Replica

Operation	Estimated Throughput	What Changed
Send money	~450 req/sec	3× Wallet Service instances, still one write DB
Check balance	~3000 req/sec	Read replica absorbs balance reads
Login	~1500 req/sec	3× Account Service instances
OTP requests	~500 req/sec	3× OTP workers consuming queue

The hard limit at scale is the PostgreSQL write primary for send money. A single Postgres instance can sustain approximately 2,000–5,000 write transactions per second with proper indexing and connection pooling (PgBouncer). Beyond that, database sharding or a purpose-built financial ledger becomes necessary — that is genuine large-scale fintech territory, well beyond this system's scope.

Summary: This architecture comfortably serves 50,000–100,000 daily active users on a single VPS.
With horizontal scaling (3 instances + read replica), it handles ~500,000 DAU.
The send-money operation is always the bottleneck — it serializes at the DB level by design.
That serialization is the price of ACID correctness. It is the right tradeoff for a financial system.

11. Closing Notes

This architecture applies the following industry principles deliberately:

- Service boundaries drawn on independent failure domains, not arbitrary decomposition
- Database per service enforced via Postgres schema + credential isolation
- JWT stateless verification at each service — User Service not in critical payment path
- Atomic financial transactions with SELECT FOR UPDATE — correctness over throughput
- Async OTP delivery via RabbitMQ — decoupled from synchronous login path
- Layered security — transport, session, rate limiting, input validation, DB constraints
- Two-tier logging — audit (permanent, consistent) vs operational (ephemeral, async)
- Full observability stack — uptime, errors, metrics, real-time system state

And the following over-engineering explicitly avoided:

- Kubernetes — Docker Compose is the correct tool for this team size and service count
- Separate auth service — no cross-application SSO requirement exists
- Split wallet and transaction services — would require distributed transactions
- Service mesh — adds operational complexity with no benefit at 3 services
- Multiple RabbitMQ nodes — single durable node is sufficient

The goal of this document is to demonstrate that good architecture is not about using every available tool — it is about choosing the right tool for the right problem, and being able to justify every decision.